

Rapport Scientifique : Analyse de Sentiments sur les Réseaux Sociaux

Salma Kaiss (2025)

December 9, 2025

1 Introduction

1.1 Contexte et Problématique

L'analyse des sentiments sur les réseaux sociaux est une tâche essentielle pour comprendre les opinions publiques et la perception des utilisateurs vis-à-vis de différents sujets. Cette analyse peut être utilisée pour des applications variées, telles que la modération de contenu, le marketing et la veille d'opinion. Dans ce projet, l'objectif est de prédire le sentiment des messages provenant des utilisateurs sur les réseaux sociaux en utilisant des techniques de machine learning. Le problème posé est une **classification multi-classe** des sentiments des messages dans trois catégories : **positif**, **neutre** et **négatif**.

1.2 Objectifs

Le principal objectif de ce projet est de développer un modèle capable de classer automatiquement le sentiment d'un message sur les réseaux sociaux. Nous testerons plusieurs algorithmes de machine learning et nous évaluerons leur performance selon des métriques telles que l'**accuracy**, le **F1-score**, et la **matrice de confusion**.

2 Sélection du Dataset

2.1 Source et Description

Le dataset utilisé provient de Kaggle et peut être trouvé sous le lien suivant : <https://www.kaggle.com/code/alkidiarete/social-media-analysis-sentiment>.

Ce jeu de données contient des messages provenant de réseaux sociaux étiquetés selon leur sentiment (positif, neutre, négatif). Le fichier CSV contient 800 exemples, répartis entre trois classes différentes : **positif**, **neutre** et **négatif**.

2.2 Dictionnaire des Données

Le dataset contient les variables suivantes :

- Nombre d'exemples : 800
- Nombre de classes : 3
 - Positif : Messages ayant un sentiment positif.
 - Neutre : Messages sans sentiment particulier.
 - Négatif : Messages ayant un sentiment négatif.
- Features principales :
 - Text : Texte du message (chaîne de caractères).
 - Sentiment : Cible à prédire (positive, neutral, negative).

3 Implémentation Technique (Python)

Le code source utilisé pour cette analyse est disponible sur GitHub. Voici les principales étapes suivies dans l'implémentation du modèle :

3.1 Pré-traitement des Données

- Chargement des données depuis un fichier CSV.
- Nettoyage des données :
 - Suppression des doublons.
 - Retrait des valeurs manquantes.
 - Nettoyage du texte (suppression des caractères spéciaux, conversion en minuscules).
- Encodage des variables catégorielles avec **Label Encoding**.
- Tokenisation et vectorisation avec **TF-IDF**.
- Normalisation et standardisation des données (non nécessaire ici).

3.2 Analyse Exploratoire des Données (EDA)

Des visualisations ont été créées pour observer la distribution des sentiments et la longueur des messages :

- Histogrammes pour les sentiments.
- Boxplots pour la longueur des messages.
- Heatmap pour étudier la corrélation des variables.

3.3 Feature Engineering

Des nouvelles variables ont été créées pour améliorer les performances du modèle :

- Longueur du texte.
- Nombre de mots dans chaque message.

3.4 Modélisation

Trois modèles de machine learning ont été testés :

- Logistic Regression
- Support Vector Classifier (SVC)
- Random Forest Classifier

La validation croisée et l'optimisation des hyperparamètres ont été réalisées avec **GridSearchCV**. Les performances ont été évaluées avec **accuracy**, **F1-score** et la **matrice de confusion**.

4 Résultats et Discussion

4.1 Performances des Modèles

Les performances des modèles sont comparées dans le tableau ci-dessous :

Le modèle **Random Forest** a montré les meilleures performances, avec une **accuracy** de 87.2% et un **F1-score** de 0.84.

Algorithme	CV-Score ($\mu \pm \sigma$)	Accuracy Test	F1-Macro
Logistic Regression	0.78 ± 0.03	0.79	0.76
Random Forest	0.85 ± 0.02	0.872	0.84
SVC	0.82 ± 0.04	0.83	0.80

Table 1: Comparaison des performances des modèles

4.2 Matrice de Confusion

Voici la matrice de confusion pour le modèle Random Forest :

$$C = \begin{bmatrix} 123 & 18 & 9 \\ 15 & 360 & 25 \\ 10 & 30 & 210 \end{bmatrix}$$

Les erreurs principales sont des confusions entre les classes **Neutre** et **Positif**.

4.3 Analyse des Erreurs

Les erreurs les plus courantes concernent la confusion entre les messages **Neutres** et **Positifs**, ce qui peut être dû à des ambiguïtés dans le langage.

5 Conclusion et Perspectives

5.1 Limites du Modèle

Quelques limites à considérer :

- **Déséquilibre des classes** : La classe neutre domine largement, ce qui peut affecter les résultats pour les autres classes.
- **Perte d'information sémantique** : Le modèle n'exploite pas la structure sémantique du texte. Des modèles comme **BERT** pourraient améliorer les performances.
- **Temporalité** : Les informations de timestamp ne sont pas prises en compte.

5.2 Perspectives d'Amélioration

- **Rééquilibrage des classes** : Utilisation de techniques comme SMOTE ou des poids de classe pour améliorer les performances sur les classes minoritaires.
- **Modèles NLP avancés** : Intégration de modèles comme **BERT** pour une meilleure capture des relations sémantiques.
- **Exploitation de la temporalité** : Prise en compte des informations temporelles des messages pour améliorer l'analyse des tendances.
- **Déploiement en temps réel** : Création d'une API avec FastAPI pour la modération de contenu en temps réel.

6 Références

1. Kaiss, S. (2025). *salma_kaiss_ipynb.py*.
2. Nedioui, M.A. (2021). *Thèse Analyse des Sentiments*, Univ. Biskra.
3. Menasri, A. (2023). *Analyse Sentiments Arabe Dialectal*, Univ. Guelma.

7 Annexe : Code source complet

<https://github.com/SalmaKaiss-ui/Data-Science/blob/main/Social>